

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO



FACULTAD DE CIENCIAS DE LA COMPUTACIÓN Y DISEÑO DIGITAL CARRERA DE INGENIERÍA EN SOFTWARE

TEMA:

GA – EXPOSICIÓN SEGUNDO CORTE: HERRAMIENTAS CASE (VISUAL PARADIGM)

MATERIA:

INGENIERÍA EN REQUERIMIENTOS

DOCENTE:

ING. GUERRERO ULLOA GLEISTON CICERÓN, PhD.

INTEGRANTES:

FUERTES ARRAES EDSON DANIEL, CI: 0929115087, Correo: efuertes2@uteq.edu.ec

CEDEÑO AVILA WINSTON DAMIAN, CI: 0942833492, Correo: wcedenoa2@uteq.edu.ec

CEDEÑO CORONADO WILSON LIZANDRO, CI: 1206867200, Correo: wcedenoc2@uteq.edu.ec

EQUIPO G

CURSO:

CUARTO SEMESTRE INGENIERÍA EN SOFTWARE “B”

SISTEMA PROYECTO FIN DE CURSO:

MUNDIPETS (DANIEL FUERTES - TITULAR PFC))

Repositorio GitHub:

<https://github.com/MiloSaurio4kHD/PFC-MundiPets-MDD-EquipoG.git>

QUEVEDO – LOS RÍOS – ECUADOR

2026 – 2027 PPA

Índice

1.	Resumen	4
2.	Marco teórico	4
3.	Justificación de la herramienta MDD seleccionada	4
3.1.	Criterios de selección	4
3.2.	El motor Instant Generator	4
3.3.	Comparación con una alternativa descartada: Enterprise Architect	5
3.4.	Aplicación a MundiPets	6
4.	Descripción del subsistema del PFC modelado	7
5.	Modelo UML de origen	7
5.1.	Diagrama de clases	7
5.2.	Diagramas complementarios (si aportan)	7
5.3.	Perfiles y estereotipos aplicados	7
6.	Configuración del mecanismo de generación	7
7.	Proceso de generación de código	7
8.	Verificación del código generado	7
9.	Cierre del ciclo (roundtrip) y trazabilidad	7
10.	Reparto del trabajo por integrante	7
11.	Conclusiones	7
12.	Referencias	7
	Anexos	7

Índice de figuras

Índice de tablas

1 Resumen

2 Marco teórico

3 Justificación de la herramienta MDD seleccionada

3.1 Criterios de selección

Para representar el ciclo de desarrollo dirigido por modelos en el subsistema de MundiPets, el equipo optó por trabajar con Visual Paradigm como herramienta CASE. La decisión no fue arbitraria: se basó en tres puntos concretos, qué tantos lenguajes de programación cubre su generador, si se pueden ajustar las plantillas con las que produce el código, y cuánto cuesta adoptarla frente a otras opciones parecidas que también sirven para un trabajo de este nivel.

Lo primero, la variedad de lenguajes que el generador soporta, terminó pesando bastante porque, para cuando tocaba preparar la exposición, todavía no se había cerrado del todo qué tecnología se usaría en la implementación final del PFC. Trabajar con una herramienta que admite varios lenguajes de salida evita rehacer el modelado si, ya avanzado el proyecto, el equipo termina programando en algo distinto a lo mostrado durante la demo. Ningún CASE tool es el único con esta ventaja, pero entre las opciones que se revisaron, Visual Paradigm fue de las que más lenguajes ofrecía sin necesidad de pagar licencias adicionales.

El segundo punto, poder editar las plantillas de generación, importaba porque la idea no era simplemente sacar clases vacías del diagrama, sino un código que ya se pareciera, al menos un poco, a cómo el grupo suele escribir sus proyectos. Con un generador cerrado, casi toda la salida termina reescribiéndose a mano para que encaje con el estilo que se busca, y ahí se pierde buena parte del sentido de automatizar nada. Visual Paradigm deja tocar la plantilla directamente, así que ese trabajo de ajuste se hace una sola vez y sirve para todas las generaciones que vengan después.

El último criterio, el costo, tenía que ver con que este es un proyecto de curso y nadie del equipo va a pagar una licencia comercial. Por eso se prefirieron herramientas con versión gratuita o barata que alcanzara para modelar en UML y generar el código que se necesita para la demo, sin quedar atados a una prueba temporal que se venza justo antes de la fecha de exposición. En ese sentido, Visual Paradigm cumple con este criterio porque ofrece una edición Community gratuita para usos no comerciales, entre ellos proyectos educativos y personales [1].

Antes de quedarse comparando solo Visual Paradigm contra Enterprise Architect, el equipo también le echó un ojo, de forma más rápida, a StarUML y a ArgoUML. Ninguna de las dos pasó el primer filtro: StarUML porque su generación de código cubre menos lenguajes de los que se necesitaban, y ArgoUML porque el proyecto parece tener poco movimiento reciente, lo que no da mucha confianza para depender de él en una entrega. Con eso quedaron solo dos candidatas serias, Visual Paradigm y Enterprise Architect, las únicas con un módulo de generación completo como para sostener la demostración.

Los tres criterios, eso sí, no pesaron igual. La cobertura de lenguajes y el costo funcionaron casi como filtros de entrada, cosas que cualquier herramienta tenía que cumplir para seguir en la carrera, mientras que poder personalizar las plantillas terminó siendo el que desempató entre las dos que sí pasaban los primeros dos filtros. Y ahí es donde Visual Paradigm le ganó a Enterprise Architect: ambas cumplían razonablemente bien lo primero, pero se notaba más diferencia en el tercer punto.

3.2 El motor Instant Generator

Dentro de Visual Paradigm existe un módulo llamado Instant Generator que se encarga de convertir un diagrama de clases UML en código. Puede generar, entre otros, Java, C#, C++, Python, PHP, VB.NET y plantillas pensadas para Hibernate, y también hace el camino contrario: reconstruir el modelo UML a partir de código que ya existe [2]. Esa cobertura amplia le sirve a MundiPets porque el subsistema modelado se puede generar en el lenguaje que finalmente se defina para el PFC sin tener que cambiar de herramienta más adelante.

Otra cosa que pesó en la elección es que Instant Generator no depende de un motor cerrado para producir el código: sus plantillas están hechas sobre Apache Velocity, que es un motor de plantillas abierto y editable [3]. Apache Velocity, de hecho, es un proyecto independiente de código abierto de la Apache Software Foundation, lo que explica por qué sus plantillas pueden modificarse libremente en vez de quedar cerradas dentro del generador [4]. Gracias a eso se puede ajustar cómo quedan formateadas las clases generadas, dónde van las llaves o si se agregan comentarios de cabecera, algo que sirve si el equipo necesita que el código generado siga alguna convención particular del PFC.

En la práctica, para generar código hace falta tener listo el diagrama de clases dentro de Visual Paradigm. Desde ahí se va a Tools > Code > Instant Generator, se elige el lenguaje de salida, se indica dónde debe quedar el código y se marcan las clases del subsistema que se quiere traducir. El motor recorre cada clase y convierte los atributos en campos o propiedades según el lenguaje elegido, las operaciones en métodos vacíos que el equipo tiene que completar después, y las relaciones (asociación, agregación, composición, herencia) en su equivalente dentro del lenguaje, por ejemplo, colecciones cuando la multiplicidad es uno a muchos, o clases hijas cuando hay generalización.

Como las plantillas están sobre Velocity, cada parte del archivo generado (encabezado, declaración de la clase, getters y setters, comentarios) es un fragmento independiente que se puede modificar sin tocar el resto del generador. Eso quiere decir que si el equipo quiere, por ejemplo, que cada clase salga con un comentario indicando quién es el responsable, o con alguna anotación de persistencia en los atributos, ese cambio se hace una vez en la plantilla y queda aplicado para todas las generaciones futuras, sin repetirlo clase por clase.

El módulo también funciona al revés: puede tomar código ya escrito y reconstruir el diagrama de clases correspondiente. Esa función no se va a usar en la demo de MundiPets, pero se tuvo presente al elegir la herramienta como una opción de respaldo, por si en una etapa posterior del PFC hace falta documentar en UML algo que ya se empezó a programar directamente.

Para dar un ejemplo concreto del subsistema, la clase `HistorialMedico`, con atributos como la fecha de creación, las observaciones generales y la referencia a la `Mascota`, quedaría generada con esos mismos campos ya declarados según la sintaxis del lenguaje elegido, más sus métodos de acceso hechos automáticamente. La relación uno a uno con `Mascota` aparecería como una simple referencia a un objeto `Mascota` dentro de la clase generada, y una operación que el equipo haya puesto en el diagrama, como registrar una nueva entrada en el historial, saldría como un método sin cuerpo, listo para que se le agregue la lógica durante el desarrollo del PFC.

También se revisó cómo queda organizada la salida. El generador crea un archivo por clase y respeta la estructura de paquetes o espacios de nombres definida en el propio modelo; si el equipo agrupó las clases del subsistema bajo un paquete, digamos, gestión de perfil médico, esa agrupación se refleja igual en la carpeta generada, en vez de tirar todos los archivos sueltos en un mismo lugar. Esto evita tener que reordenar manualmente los archivos antes de meterlos al proyecto del PFC.

3.3 Comparación con una alternativa descartada: Enterprise Architect

Como alternativa se revisó Enterprise Architect, de Sparx Systems, una herramienta CASE bastante usada para modelado empresarial y arquitectura de sistemas. También soporta generación e ingeniería inversa de código, y tiene buena fama en el modelado de arquitecturas complejas y proyectos grandes.

Aun así, se descartó por dos razones principales. La primera es que su curva de aprendizaje es más pronunciada que la de Visual Paradigm, porque está pensada para arquitectos empresariales con flujos de trabajo más largos, algo que no calza con el tiempo disponible para una exposición de corte parcial. La segunda es que configurar la generación de código en Enterprise Architect no resulta tan intuitivo para alguien que recién empieza, mientras que en Visual Paradigm todo el flujo (Tools > Code > Instant Generator) permite elegir lenguaje, ruta de salida y clases desde una sola ventana, lo que hace más simple la demostración en vivo prevista para el cierre.

Más allá de la curva de aprendizaje y la configuración, también se comparó la documentación disponible para quien recién usa la herramienta. Visual Paradigm tiene guías paso a paso específicas para Instant Generator, con capturas del proceso completo, lo cual ahorró tiempo al equipo durante la preparación. La de Enterprise Architect, en cambio, viene mezclada con la documentación de otras funciones de arquitectura empresarial, así que encontrar justo la parte de generación de código tomó más tiempo de búsqueda.

Otro punto fue el licenciamiento. Enterprise Architect ofrece una prueba de treinta días para quien no tiene licencia, después de la cual hay que comprar alguna de sus ediciones pagas [5], lo que trae el riesgo de que la prueba se venza antes de la exposición si no se coordinan bien los tiempos. Visual Paradigm, en cambio, tiene una edición gratuita para modelado UML básico, que alcanza para modelar el subsistema de MundiPets y completar el ciclo de generación sin depender de una fecha de vencimiento.

Con todo esto, el equipo decidió que, aunque Enterprise Architect es una herramienta sólida para proyectos más grandes, Visual Paradigm se ajusta mejor al alcance y al tiempo que hay para esta exposición en particular, sin que eso signifique descartarla para etapas más adelante del PFC donde el modelo sea más complejo.

También se revisó qué tan fácil es exportar el diagrama como apoyo para el informe escrito del PFC, aparte del código generado. Visual Paradigm permite exportar los diagramas en formatos de imagen comunes y generar un

documento de especificación a partir del modelo, lo que ayuda a mantener coherencia entre lo que se muestra en la exposición y lo que se entrega por escrito. Enterprise Architect tiene algo similar, pero pensado para reportes más largos y con más configuración de la que hace falta para una entrega de corte parcial, así que en este caso se vio como una ventaja menor frente al flujo más directo de Visual Paradigm.

3.4 Aplicación a MundiPets

El equipo limitó la demostración al subsistema de gestión del perfil de mascota y validación de información médica, con entidades como `Mascota`, `HistorialMedico`, `DocumentoVeterinario`, `CarnetVacunacion`, `Vacuna`, `ValidacionMedica` y `AntecedenteGenetico`. La elección de Visual Paradigm tiene sentido justo con este subsistema porque, al correr Instant Generator sobre su diagrama de clases, se puede mostrar de forma directa cómo cada clase, atributo y operación del modelo termina en código ejecutable, sin escribir el esqueleto de las clases a mano. Esto se ve especialmente bien en relaciones como `Mascota`–`HistorialMedico` (uno a uno) o `Mascota`–`DocumentoVeterinario` (uno a muchos), donde la manera en que Instant Generator traduce esa cardinalidad es, en sí, parte de lo que se busca mostrar.

La demostración en vivo va a seguir una secuencia corta: primero se muestra el diagrama de clases completo del subsistema en Visual Paradigm, señalando los atributos y operaciones de cada entidad; después se corre Instant Generator sobre ese diagrama, escogiendo el lenguaje y la ruta de salida delante del público; y al final se abre el código generado para mostrar, clase por clase, cómo quedaron los atributos, las operaciones y las relaciones del modelo. La idea de esta secuencia es que la audiencia vaya conectando cada parte del diagrama con su equivalente en el código, en vez de ver el código generado como algo aislado.

En concreto, la relación `Mascota`–`HistorialMedico`, que es uno a uno, sirve para mostrar cómo Instant Generator convierte esa cardinalidad en una referencia directa entre las clases generadas, mientras que `Mascota`–`DocumentoVeterinario`, que es uno a muchos, termina como una colección dentro de la clase `Mascota`. Poner ambos casos en la misma demo ayuda a que se note que el motor no genera lo mismo para toda asociación, sino que adapta la estructura según la multiplicidad que tenga cada relación en el modelo. Esa misma lógica se aplicaría después a `CarnetVacunacion` con `Vacuna` y a `ValidacionMedica` con `AntecedenteGenetico`, aunque estas dos quedan fuera del recorrido principal que se va a mostrar en el cierre.

Sobre `ValidacionMedica` puntualmente, se decidió no meterla en el recorrido principal porque buena parte de lo que hace esta clase tiene que ver con reglas de negocio, por ejemplo, revisar que un carnet de vacunación corresponda a vacunas puestas dentro del rango de edad esperado según la especie de la mascota, y ese tipo de validación no se ve reflejada en el diagrama de clases como tal, sino que se programa después de generar el código, dentro de los métodos que Instant Generator deja vacíos. Comentar esto durante la exposición ayuda a que quede claro qué resuelve la generación automática y qué sigue dependiendo de que el equipo programe manualmente.

Para la parte en vivo, se repartieron roles concretos: una persona va a manejar el teclado y ejecutar los pasos dentro de Visual Paradigm, y otra va a ir narrando en paralelo qué hace cada paso y por qué se relaciona con lo explicado antes. Esto es para que quien maneja la herramienta no tenga que parar a explicar cada clic, algo que suele alargar de más este tipo de demostraciones.

4 Descripción del subsistema del PFC modelado

5 Modelo UML de origen

5.1 Diagrama de clases

5.2 Diagramas complementarios (si aportan)

5.3 Perfiles y estereotipos aplicados

6 Configuración del mecanismo de generación

7 Proceso de generación de código

8 Verificación del código generado

9 Cierre del ciclo (roundtrip) y trazabilidad

10 Reparto del trabajo por integrante

11 Conclusiones

12 Referencias

- [1] Visual Paradigm International. “Community Edition — Free UML Modeling Software,” visitado 15 de jul. de 2026. dirección: <https://www.visual-paradigm.com/editions/community/>
- [2] Visual Paradigm International. “UML/Code Generation Software — Instant Generator,” visitado 15 de jul. de 2026. dirección: <https://www.visual-paradigm.com/features/code-engineering-tools/>
- [3] Visual Paradigm International. “Instant Generation in Visual Paradigm — User’s Guide, Part IX, Chapter 2,” visitado 15 de jul. de 2026. dirección: https://www.visual-paradigm.com/support/documents/vpuserguide/276/330_instantgener.html
- [4] The Apache Software Foundation. “Apache Velocity Engine,” visitado 15 de jul. de 2026. dirección: <https://velocity.apache.org/engine/index.html>
- [5] Sparx Systems. “Enterprise Architect — Free Trial,” visitado 15 de jul. de 2026. dirección: <https://sparxsystems.com/products/ea/trial/request.html>

Anexos

Estructura del repositorio GitHub

```
PFC-MundiPets-MDD-EquipoG/  
  README.md  
  docs/  
    informe.tex  
    informe.pdf  
    referencias.bib  
    figuras/  
  modelo/  
    <proyecto_nativo>.vpp  
  perfiles/  
  plantillas/  
  generado/  
  evidencias/  
    capturas/  
    video-demo.mp4  
  exposicion/  
    diapositivas.pdf
```

Anexos adicionales

Tabla de mapeo extensa (modelo → código)

Listados de código representativo (fragmentos)

Instrucciones detalladas de reproducibilidad